

Ingeniería de Software II

Estimación de proyectos de software

Actividad – Estimación

Estimar sin utilizar ningún tipo de información externa, ni googleo ni nada más que la intuición

Descripción	Rango
Temperatura en la superficie del sol	
Latitud de Shanghai	
Área de Asia	
Año de nacimiento de Alejandro Magno	
Cantidad de dólares en el mundo en 2004	
Recaudación mundial de Titanic	
Cantidad de libros publicados en USA desde 1800	
Peso record de una ballena azul	
Tamaño del salto más alto en Iguazú	
Longitud de la costa atlántica de Argentina	

Actividad – Estimación

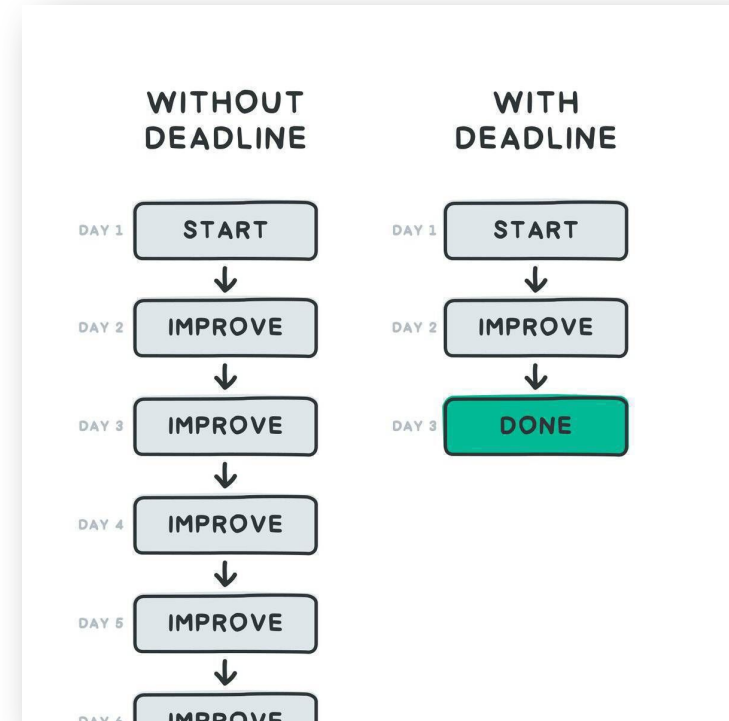
Descripción	Valor
Temperatura en la superficie del sol	6000°C
Latitud de Shanghai	31°N
Área de Asia	44390000 km ²
Año de nacimiento de Alejandro Magno	356 AC
Cantidad de dólares en el mundo en 2004	719.000.000.000
Recaudación mundial de Titanic	1.836.000.000
Cantidad de libros publicados en USA desde 1800	22 millones
Peso record de una ballena azul	170000 kg
Tamaño del salto más alto en Iguazú	82 mt
Longitud de la costa atlántica de Argentina	4989 mt

Actividad – Estimación

- ▶ ¿Harían un proyecto estimado por ustedes?
- ▶ ¿Eligieron valores absolutos o rangos? → Ojo con la presión por acotar los rangos de estimación.
- ▶ Tener seguridad también es poder asumir cuando “no se tiene idea” (¿y los supuestos?)

Ley de Parkinson

- Estimamos porque necesitamos tener una noción del tamaño de la solución del requerimiento.
- Ley de Parkinson:
 - Cyril Parkinson (1957): "el trabajo se expande hasta llenar el tiempo disponible para su realización".
 - Servicio Civil Británico: a pesar de la decadencia del Imperio Británico, la cantidad de empleados aumentaba continuamente.
 - Burocracia: Los funcionarios multiplican subordinados. Los funcionarios se crean trabajo entre sí.
 - Crecimiento Administrativo: El número de empleados en una burocracia aumenta alrededor de un 5-7 % por año, independientemente de la carga de trabajo real.



Ley de Parkinson

- Ejemplos:
 - "Los datos se expanden hasta llenar el espacio disponible para su almacenamiento"
- Tres Leyes Fundamentales de Parkinson:
 - El trabajo se expande hasta llenar el tiempo disponible.
 - Los gastos aumentan hasta cubrir todos los ingresos.
 - "El tiempo dedicado a un tema es inversamente proporcional a su importancia" (Ley de la trivialidad, también de C. Parkinson).

Ley de Parkinson

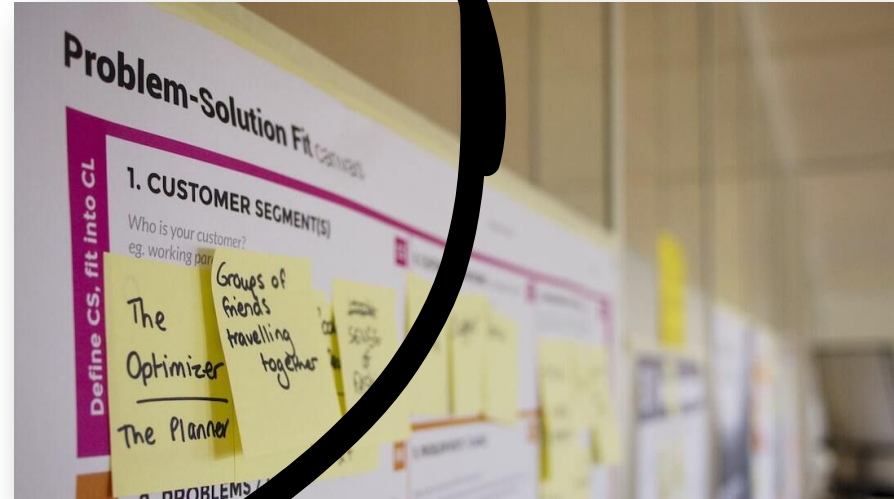
"El trabajo se expande hasta llenar el tiempo disponible para que se termine"



- 1 Cuando hay poco tiempo para hacer un trabajo, el esfuerzo es muy grande
- 2 A medida que aumenta el tiempo disponible para hacer el mismo trabajo, el esfuerzo se va haciendo cada vez menor = pérdida de tiempo

Puntos de Caso de Uso

- **Definición:** Método de estimación basado en la complejidad y número de casos de uso en un proyecto. Ayuda a evaluar el tamaño del software y el esfuerzo necesario.
- **Proceso:** Se asigna una ponderación a cada caso de uso según su complejidad, sumando estos valores para obtener la estimación total del proyecto.
- **Ventajas:** Proporciona una visión clara del alcance del proyecto y facilita la comunicación entre desarrolladores y stakeholders.



Puntos de Caso de Uso



Antes de seguir: busquen en su LLM amigo qué es y porque es importante el “factor de productividad” para una estimación por Puntos de Caso de Uso y otros métodos de estimación.

Puntos de Caso de Uso

Ponderación de Factores Ambientales y Técnicos:

Environmental Factor		Multiplier	Relative Magnitude (Enter 0-5)	Description
1	Familiarity With The Project	1,5		How much experience does your team have working in this domain? The domain of the project will be a reflection of what the software is intended to accomplish, not the implementation language. In other words, for an insurance compensation system written in java, you care about the team's experience in the insurance compensation space - not how much java they've written. Higher levels of experience get a higher number.
2	Application Experience			How much experience does your team have with the application. This will only be relevant when making changes to an existing application. Higher numbers represent more experience. For a new application, everyone's experience will be 0.
3	OO Programming Experience	1		How much experience does your team have at OO? It can be easy to forget that many people have no object oriented programming experience if you are used to having it. A user-centric or use-case driven project will have an inherently OO structure.
4	Lead Analyst Capability	0,5		How knowledgeable and experienced is your lead analyst? Do you have a Standish Group skill and knowledge level of 1 or 2?
5	Motivation	1		How motivated is your team? A motivated team will work harder and faster to complete a project.
6	Stable Requirements	2		Changes in requirements are a major source of project failure. If you have a system for managing those changes, the multiplier for this factor will be 1. Note, the multiplier for this factor will be 1 if you have a system for managing those changes.
7	Part Time Staff	-1		Note, the multiplier for this factor will be 1 if you have a system for managing those changes. If you have part-time staff, the multiplier will be -1. If you have full-time staff, the multiplier will be 0.
8	Difficult Programming Language	-1		This multiplier is also negative. If you are using a difficult programming language, the multiplier will be -1. If you are using a familiar programming language, the multiplier will be 0. Java might be difficult for some people, but it is familiar for many.
Calculated EF			1,4	

Technical Factor		Multiplier	Relative Magnitude (Enter 0-5)	Description
1	Distributed System Required	2		Is the architecture of the solution may be centralized or single-tenant, or it may be distributed (like an n-tier solution) or multi-tenant. Higher numbers represent a more complex architecture.
2	Response Time Is Important	1		The quickness of response for users is an important (and non-trivial) factor. For example, if the server load is expected to be very low, this may be a trivial factor. Higher numbers represent increasing importance of response time (a search engine would have a high number, a daily news aggregator would have a low number).
3	End User Efficiency			Is the application being developed to optimize on user efficiency, or just capability? Higher numbers represent projects that rely more heavily on the application to improve user efficiency.
4	Complex Internal Processes	1		Is there a lot of difficult algorithmic work to do and test? Complex algorithms (resource leveling, time-domain systems analysis, OLAP cubes) have higher numbers. Simple database queries would have low numbers.
5	Reusable Code	1		Is heavy code reuse an objective or goal? Code reuse reduces the amount of effort required to deploy a project. It also reduces the amount of time required to debug a project. A shared library function can be re-used multiple times, and fixing the code in one place can resolve multiple bugs. The higher the level of re-use, the lower the number.
6	Installation Ease	0,5		Is ease of installation for end users a key factor? The higher the level of competence of the users, the lower the number.
7	Usability			Is ease of use a primary criteria for acceptance? The greater the importance of usability, the higher the number.
8	Cross-Platform Support	2		Does the customer require the ability to change or customize the application in the future? The more change / customization that is required in the future, the higher the value.
9	Easy To Change	1		Do you have to address database locking and other concurrency issues? The more attention you have to spend to resolving conflicts in the database, the higher the value.
10	Highly Concurrent	1		Can existing security solutions be leveraged, or must custom code be developed? The more custom security work you have to do (field level, page level, etc.), the higher the value.
11	Custom Security	1		Will the application require the use of third party controls or libraries? Like re-usable code, third party code can reduce the effort required to deploy a solution. The more third party code (and the more reliable the third party code), the lower the number.
12	Dependence On Third-Party Code	1		How much user training is required to use the application complex, or supporting complex activities? The longer it takes users to cross the skill threshold (achieve a level of mastery), the higher the value.
13	User Training	1		
Calculated TCF			0,6	

Puntos de Caso de Uso

Ponderación de Casos de Uso y Actores

Unadjusted Use Case Points		Multiplier	Number of Use Cases	Description
1	Simple	5	0	Simple Use Case - 1 to 3 transactions.
2	Average	10	0	Average Use Case - 4 to 7 transactions.
3	Complex	15	0	Complex Use Case - more than 7 transactions.
Calculated UUCP			0	
Insert additional rows above this row and copy the cell values to automatically update the counts of use cases by type				

Individual Use Cases		Multiplier	Number of Use Cases	Description
1	Simple	5		
2	Average	10		
3	Complex	15		
4	Simple	5		
5	Simple	5		
6	Simple	5		
7	Simple	5		
8	Simple	5		
Insert additional rows above this row and copy the cell values to automatically update the counts of use cases by type				

Actor Summary		Multiplier	Number of Actors	Description
1	Simple	1	0	Simple actors are other systems that communicate with your software via a pre-defined API. An API could be exposed through a dll, or as a REST, SOAP, or any web-service API or remote procedure call (RPC). The key element is that you are exposing interaction with your software through a specific, well-defined mechanism.
2	Average	2	0	Average actors can either be human beings interacting in a well defined protocol, or they could be systems that interact through a more complex or flexible API.
3	Complex	3	0	The original definition of complex actors specifies that users who interact with the software through a graphical user interface are complex actors. While that is true, the same classification should apply to users who interact with the system in unpredictable ways. An AJAX interface that exposes more of the underlying application (and data stores) than would be available through a rigid protocol might introduce similar complexity.
Calculated AW			0	
Insert additional rows above this row and copy the cell values to automatically update the counts of actors by type				

Individual Actors		Multiplier	Actor Name
1	Complex	3	
2	Average	2	
3	Complex	3	
4	Simple	1	
5	Simple	1	
6	Simple	1	
7	Simple	1	
8	Simple	1	
Insert additional rows above this row and copy the cell values to automatically update the counts of actors by type			

Puntos de Caso de Uso

Cálculo final:

Calculation From Other Tabs		
TCF	Technical Complexity Factor	0,6
EF	Environmental Factor	1,4
UUCP	Unadjusted Use Case Points	0
AW	Actor Weighting	0
Calculation of Use Case Points		
UCP	Use Case Points	0,0
Calculation of Estimated Effort		
Ratio	Hours of Effort per Use Case Point	28
Hours of Effort		-

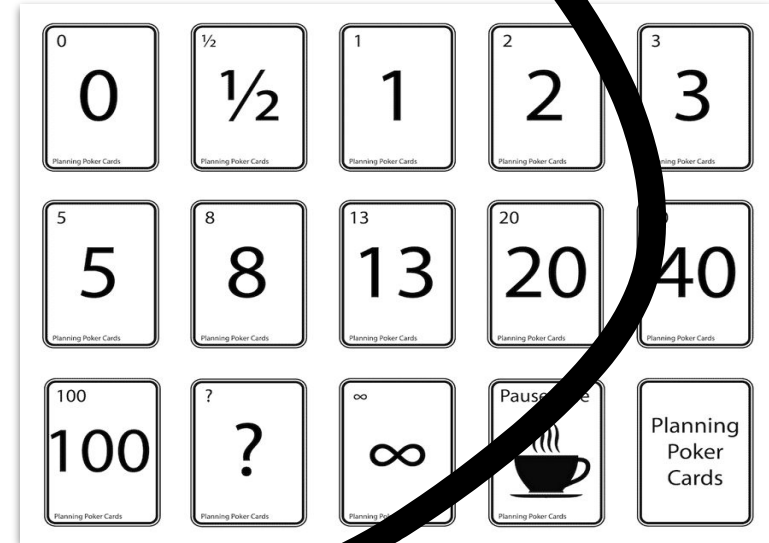
Puntos de Función

- **Definición:** Método de estimación que mide la funcionalidad entregada al usuario final. Se basa en el conteo de entradas, salidas, consultas, archivos y interfaces externas.
- **Proceso:** Se asignan valores a cada tipo de función basado en su complejidad, se suman para obtener una medida del tamaño del software.
- **Ventajas:** Proporciona una métrica objetiva y consistente para comparar proyectos y medir la productividad del desarrollo de software.

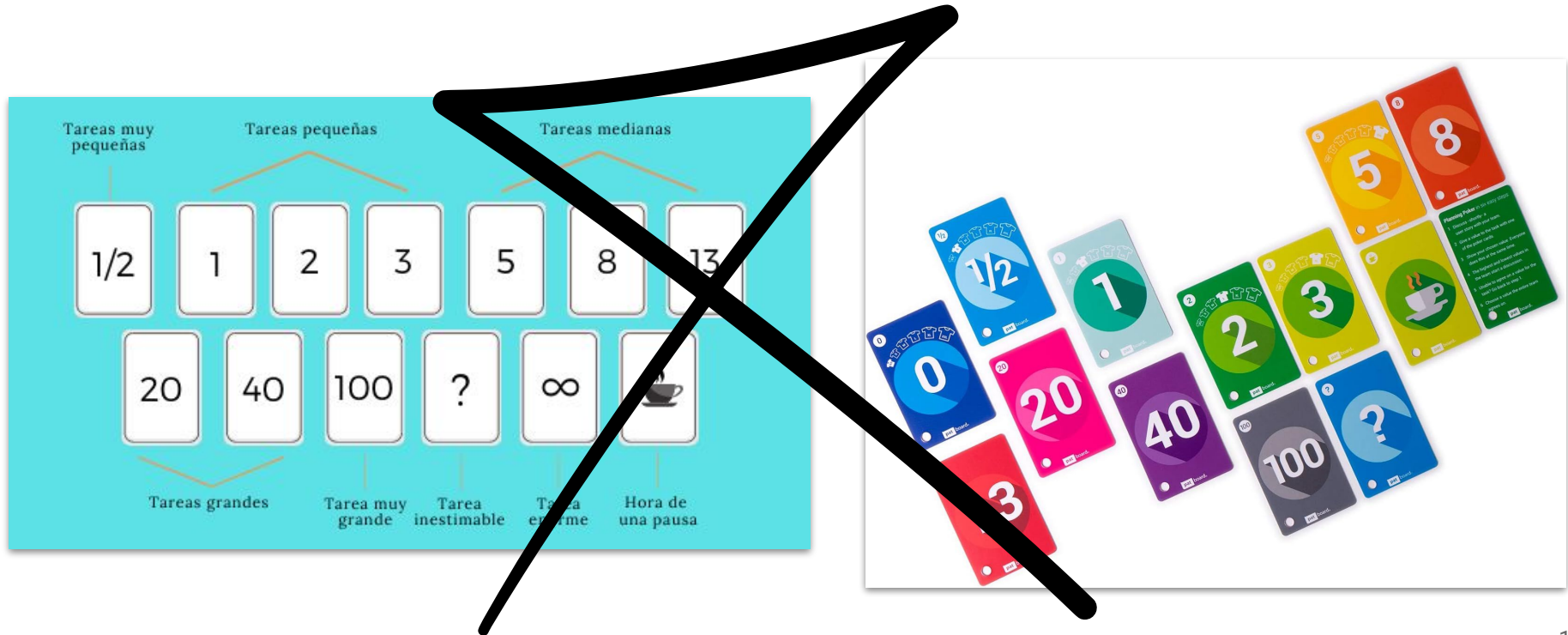


Poker Planning

- **Definición:** Método de estimación colaborativa en el que los miembros del equipo usan cartas para votar sobre el tamaño relativo de las tareas del proyecto.
- **Proceso:** Cada miembro del equipo selecciona una carta en secreto y todas las cartas se revelan al mismo tiempo. Las discusiones se centran en las diferencias de estimaciones hasta alcanzar un consenso.
- **Ventajas:** Fomenta la participación de todo el equipo, mejora la precisión de las estimaciones y ayuda a identificar riesgos y dependencias tempranas.

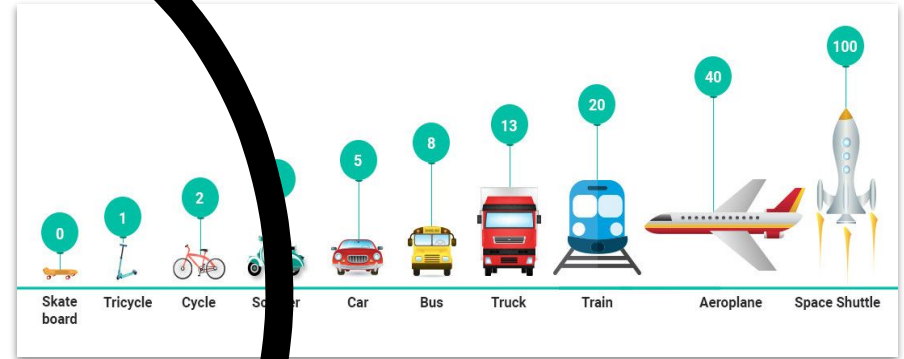


Poker Planning



Story Points

- **Definición:** Unidad de medida utilizada en metodologías ágiles para estimar el esfuerzo relativo necesario para implementar una historia de usuario.
- **Proceso:** Los equipos asignan puntos a cada historia basándose en su complejidad, cantidad de trabajo y riesgos involucrados, usando técnicas como Planning Poker.
- **Ventajas:** Facilita la planificación y priorización del trabajo, promueve la colaboración del equipo y permite un seguimiento más preciso del progreso del proyecto.



Estimación de 3 Puntos / PERT

- **Definición:** Método que utiliza tres estimaciones (optimista, pesimista y más probable) para calcular una estimación ponderada y reducir la incertidumbre.
- **PERT:** Program Evaluation and Review Technique
- **Proceso:** Se calcula la estimación final utilizando la fórmula:

$$(\text{Optimista} + (4 * \text{Más Probable}) + \text{Pesimista}) / 6$$

$$E(x) = \frac{O + 4M + P}{6}$$

- **Ventajas:** Proporciona una visión más completa de los posibles resultados y ayuda a gestionar el riesgo asociado con las estimaciones.

COCOMO

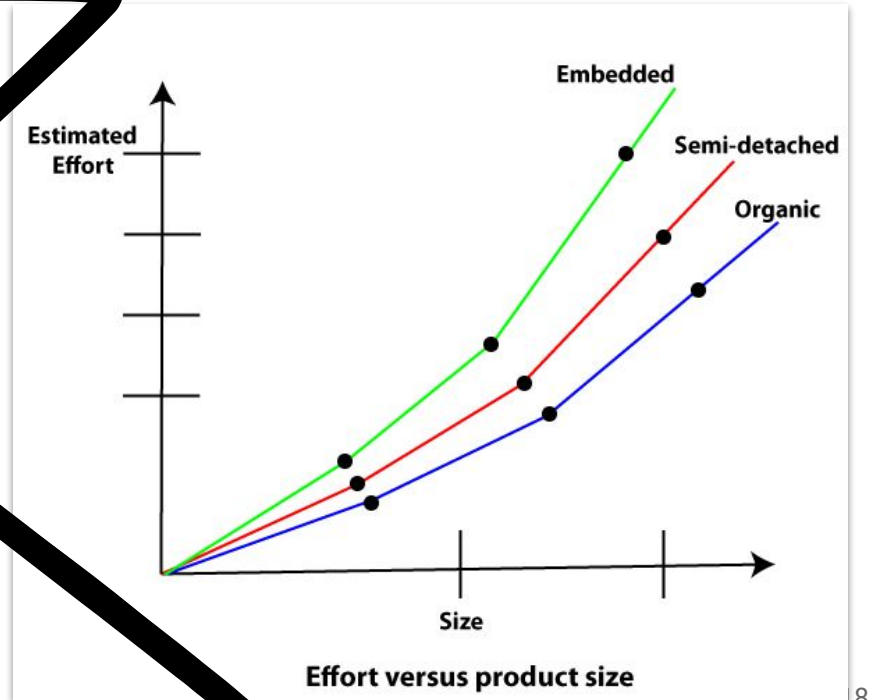
- **Definición:** Constructive Cost Model (COCOMO) es un modelo de estimación de costos de software que utiliza una fórmula basada en el tamaño del software para calcular el esfuerzo y el tiempo necesarios.
- **Variables:**
 - Tipo de proyecto (Orgánico: pequeños, equipos reducidos, requisitos claros; Semi-Detached: tamaño medio, equipos con experiencia variable; Embedded: complejos, restricciones exigentes, alto nivel de innovación)
 - Fórmula: $a \times (KLOC)^b \times F$
 - a - b dependen del tipo de proyecto
 - KLOC: cantidad de líneas (histórico)
 - F : factor de ajuste
 - Factores de Ajuste (Producto, Hardware, Personal, Proyecto)



COCOMO

- **Proceso:**

- Clasificar el proyecto según categoría (orgánico, Semi-Detached o Embedded)
- Contar o estimar KLOC (líneas de código)
- Aplicar la fórmula COCOMO usando valores de a y b apropiados
- Identificar factores de ajuste para refinar la estimación
- Calcular esfuerzo total, tiempo de desarrollo y personal requerido



Estimadores no formales

- Juicio experto
 - Un experto analiza la información disponible y emite una estimación basada en su conocimiento y experiencia.
 - Ventaja: Rápido y fácil de aplicar cuando no se tiene datos formales o históricos.
 - Desventaja: subjetivo y susceptible a sesgos personales o de grupo.
- Método Delphi / Wideband Delphi
 - Se consulta a varios expertos de manera iterativa y anónima. Cada ronda resume las estimaciones, de forma que los expertos pueden ajustar sus respuestas en función de la información recibida.
 - Ventaja: Reduce la influencia de jerarquías o liderazgos fuertes (al ser anónimo) y fomenta la convergencia de opiniones.
 - Desventaja: Puede ser costoso en tiempo si se requieren varias rondas y depende de la selección de expertos.
- Analogía con proyectos similares
 - Comparar el proyecto que se quiere estimar con otros proyectos o tareas similares previamente realizados, aprovechando la experiencia previa.
 - Ventaja: Fácil de comprender; se basa en ejemplos reales y no en suposiciones puramente teóricas.
 - Desventaja: Requiere disponer de proyectos de referencia realmente similares en cuanto a alcance, complejidad y contexto.

Estimadores no formales

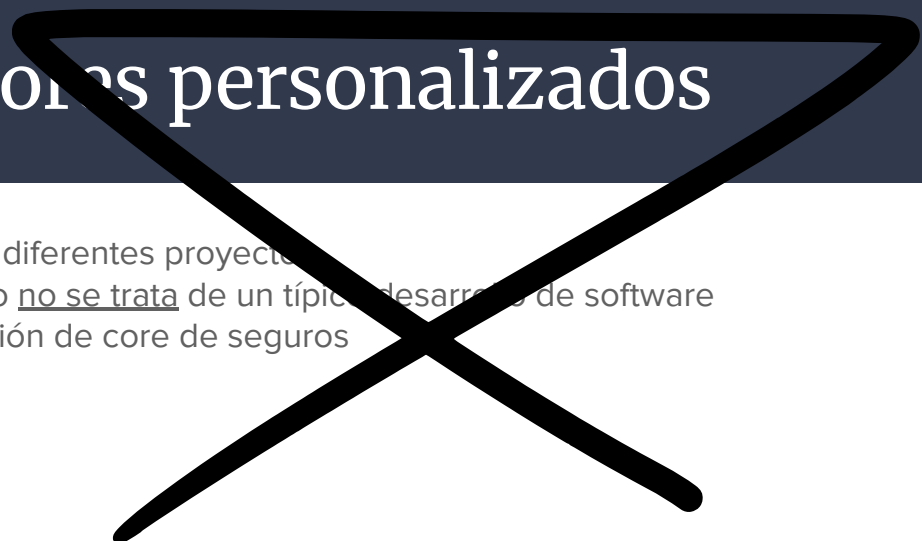
- Técnica “Talla de Camiseta” (T-Shirt Sizing)
 - Se asigna a cada elemento a estimar una “talla” (XS, S, M, L, XL) basada en criterios subjetivos de complejidad, alcance, riesgo, etc.
 - Ventaja: Es muy simple e intuitivo; bueno para estimaciones rápidas al inicio de un proyecto.
 - Desventaja: Discrepancias grandes si no se definen adecuadamente los criterios para cada “talla”.
- Estimaciones “a ojo” (estimaciones rápidas o *guesstimate*)

Se obtiene un valor aproximado rápidamente basándose en la experiencia, la lógica elemental o la información disponible en el momento, sin un método estructurado.

Ventaja: Muy veloz cuando se necesita un número aproximado de forma urgente.

Desventaja: La precisión puede ser muy baja y depender en exceso de la experiencia de quien hace la estimación.

Estimadores personalizados



- Adaptados a diferentes proyectos
- Útiles cuando no se trata de un típico desarrollo de software
- Caso: migración de core de seguros

Conceptos clave

- **Tamaño:** Se refiere a la cantidad de trabajo requerido para completar el proyecto de software, usualmente medido en líneas de código o puntos de función.
- **Esfuerzo:** Cantidad de recursos (horas de trabajo, personas, etc.) necesarios para completar el proyecto. Depende del tamaño y complejidad del proyecto.
- **Complejidad:** Grado de dificultad técnica y organizativa del proyecto. Afecta directamente al esfuerzo y tiempo necesarios para completar el proyecto.
- **Duración:** Tiempo “calendarizado” que lleva resolver un requerimiento o serie de requerimientos.

